Лабораторная работа №5. DAST и конвейер безопасности в GitHub Actions

Тема: динамический анализ работающего приложения, контроль доступа, сборка всех проверок практикума в автоматический конвейер CI. **Время:** 4 академических часа (два занятия): часть А — DAST и исправления, часть Б — GitHub Actions. **Результат:** ваш репозиторий опубликован на GitHub; на каждое изменение автоматически запускаются тесты, Gitleaks, Semgrep, Trivy и OWASP ZAP; опасное изменение конвейер останавливает.

Перед началом: выполнены ЛР1–ЛР4, установлен Docker, есть учётная запись на github.com.

---

1. Цель работы

Проверить приложение «снаружи», закрыть дефекты, которые не видят анализаторы кода, и превратить набор ручных проверок в процесс, который работает сам.

После работы вы сможете:

- объяснить, чем DAST дополняет SAST и SCA;
- провести пассивное сканирование OWASP ZAP и разобрать его отчёт;
- найти и исправить нарушение контроля доступа (IDOR);
- написать конвейер GitHub Actions с проверками безопасности и условиями приёмки;
- показать на опыте, что конвейер останавливает уязвимое изменение.

2. Теория

2.1. Что такое DAST

DAST (Dynamic Application Security Testing) — проверка **работающего** приложения. Сканер ведёт себя как очень настойчивый пользователь: открывает страницы, отправляет формы, изучает ответы сервера. Исходный код ему не нужен.

	SAST (ЛР2)	SCA (ЛР4)	DAST (ЛР5)
Что проверяет	исходный код	состав зависимостей	работающее приложение
Нужно ли запускать приложение	нет	нет	да
Что видит	опасные конструкции в коде	известные уязвимые версии	реальные ответы сервера, заголовки,

	SAST (ЛР2)	SCA (ЛР4)	DAST (ЛР5)
			поведение
Чего не видит	настройки запуска, логику доступа	ваши собственные ошибки	код и страницы, до которых не дошёл

Ни один из методов не заменяет другие. Заголовки ответа сервера видит только DAST. Неиспользуемую уязвимую библиотеку — только SCA. Опасную функцию в редко вызываемой ветке кода — только SAST.

2.2. Пассивное и активное сканирование

- **Пассивное (baseline)** — сканер обходит сайт по ссылкам и анализирует ответы, ничего не атакуя. Безопасно, занимает 1–2 минуты, подходит для каждого запуска конвейера.
- **Активное (full scan)** — сканер подставляет в параметры вредоносные значения и смотрит на реакцию. Может создавать и портить данные, работает долго.

Активное сканирование чужой системы без письменного разрешения владельца — правонарушение. В практикуме сканируется только приложение на вашем компьютере.

2.3. Как читать отчёт ZAP

ZAP делит результаты на `PASS`, `WARN` и `FAIL`. По умолчанию все находки baseline — предупреждения (`WARN`); какие из них считать провалом (`FAIL`), решаете вы в файле правил.

Два важных замечания:

- `WARN` — повод для разбора, а не доказательство уязвимости;
- `FAIL: 0` не означает «безопасно». Сканер проверил только те страницы, до которых добрался, и только от имени анонимного посетителя. Всегда смотрите, **что именно** было проверено.

2.4. IDOR — небезопасная прямая ссылка на объект

Адрес `/profile?id=2` показывает профиль номер 2. Если сервер не проверяет, **кто** спрашивает, то, заменив 2 на 1, можно прочитать чужой профиль. Это IDOR (Insecure Direct Object Reference) — частный случай нарушения контроля доступа, первой строки рейтинга OWASP Top 10.

Почему сканеры его не находят: сканер не знает, что профиль 1 «чужой». С точки зрения техники запрос корректен и ответ корректен. Нарушено бизнес-правило, а оно существует только в головах людей и в требованиях — в вашей таблице из ЛР1.

Защита — проверка на сервере при каждом запросе:

- запрос без входа в систему → код **401** (не представился);
- запрос чужого объекта → код **403** (представился, но прав нет).

2.5. Заголовки безопасности

Сервер может дать браузеру инструкции, которые ограничивают последствия атак:

Заголовок	Что делает
`Content-Security-Policy`	перечисляет, откуда странице разрешено загружать сценарии и стили; мешает XSS
`X-Content-Type-Options: nosniff`	запрещает браузеру угадывать тип файла
`X-Frame-Options: DENY`	запрещает показывать сайт во фрейме на чужой странице (защита от кликджекинга)
`Permissions-Policy`	отключает доступ страницы к камере, микрофону, геолокации

Заголовки — дополнительный рубеж. Они не заменяют исправление самой уязвимости.

2.6. CI и GitHub Actions

CI (Continuous Integration, непрерывная интеграция) — практика, при которой каждое изменение кода автоматически собирается и проверяется на сервере.

GitHub Actions — встроенная в GitHub система CI. Её основные понятия:

Понятие	Что это
Workflow (конвейер)	файл YAML в каталоге `.github/workflows/`; описывает, что и когда запускать
Event (событие)	что запускает конвейер: `push`, `pull_request`, ручной запуск
Job (задание)	группа шагов, выполняемых на одной виртуальной машине; задания идут параллельно, если не указано иное
Step (шаг)	одна команда или готовое действие
Action (действие)	готовый блок, например `actions/checkout` — получить код репозитория
Runner	виртуальная машина, на которой выполняется задание; мы используем `ubuntu-latest`
Artifact	файл, сохранённый по итогам запуска: отчёт сканера, SBOM

Главный механизм — **код завершения**. Если команда шага вернула не ноль, шаг считается проваленным, задание — красным, и GitHub помечает коммит крестиком. Именно поэтому в ЛР2–ЛР4 вы учились запускать сканеры так, чтобы находка давала ненулевой код: `--error` у Semgrep, `--exit-code 1` у Trivy.

2.7. Условия приёмки и ветки

Quality gate (условие приёмки) — правило «если проверка не пройдена, изменение не принимается».

Чтобы условие нельзя было обойти, изменения вносят не прямо в основную ветку `main`, а через **pull request (PR)** — запрос на слияние из отдельной ветки. Конвейер проверяет PR, и слияние разрешается только при зелёном результате.

2.8. Безопасность самого конвейера

Конвейер имеет доступ к коду и секретам, поэтому он сам — цель атаки. Минимальные правила:

- **наименьшие права:** `permissions: contents: read` — конвейеру достаточно читать код;
- **фиксированные версии** действий и сканеров: `actions/checkout@v4`, `aquasec/trivy:0.76.2`. В промышленных проектах действия фиксируют ещё строже — по хешу коммита;
- **секреты — только в GitHub Secrets**, никогда в тексте конвейера;
- **сбой сканера — это не «всё чисто»:** если сканер не смог отработать, проверка считается невыполненной.

Часть А. Динамический анализ и контроль доступа

Все команды выполняются в каталоге `~/devsecops-lab` с активированным окружением. В Windows (Git Bash) сначала выполните `export MSYS_NO_PATHCONV=1`.

```
cd ~/devsecops-lab
source .venv/Scripts/activate     # Windows, Git Bash
# source .venv/bin/activate       # Linux и macOS
mkdir -p reports
```

Шаг А1. Запустить цель

```
docker compose up -d --build --wait
curl http://127.0.0.1:5000/health
```

Ожидаемый результат: `{"database":"postgresql","status":"ok"}`.

Посмотрите заголовки ответа глазами сканера:

```
curl -sI http://127.0.0.1:5000/
```

Запишите, какие заголовки есть. Обратите внимание на заголовок `Server`.

Шаг А2. Пассивное сканирование ZAP

```
docker run --rm --add-host host.docker.internal:host-gateway -v "$PWD/reports:/zap/wrk/:rw" ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t http://host.docker.internal:5000 -r zap-before.html -I
```

Что означают параметры:

- `host.docker.internal` — имя, под которым контейнер видит ваш компьютер; `--add-host …` делает это имя доступным и в Linux;
- `-t` — адрес цели;
- `-r zap-before.html` — сохранить отчёт в `reports`;
- `-I` — не считать предупреждения ошибкой запуска.

Образ ZAP большой (около 2 ГБ), первая загрузка займёт несколько минут. Само сканирование — 1–2 минуты.

Ожидаемый результат — последняя строка вывода:

```
FAIL-NEW: 0  FAIL-INPROG: 0  WARN-NEW: 10  WARN-INPROG: 0  INFO: 0  IGNORE: 0  P
ASS: 58
```

Если не получилось:

- **В Linux отчёт не создаётся, ошибка `Permission denied`** — выполните `chmod 777 reports` и повторите.
- **ZAP не может подключиться к цели** — проверьте, что `curl http://127.0.0.1:5000/health` отвечает.

Откройте `reports/zap-before.html` в браузере и заполните таблицу по всем десяти предупреждениям. Первая строка — образец.

Код	Название	Что означает простыми словами	Настоящая проблема?	Что делать
10038	Content Security Policy Header Not Set	сервер не сообщает браузеру, откуда можно загружать сценарии	да	добавить заголовок
…				

Затем ответьте на два вопроса:

1. Какие адреса посетил ZAP? (Раздел с перечнем URL в отчёте.) Есть ли среди них страницы, доступные только после входа?
2. ZAP показал `FAIL: 0`. Сообщил ли он о том, что любой посетитель может прочитать профиль администратора?

В отчёт: итоговая строка, таблица предупреждений, ответы на вопросы.

Шаг А3. Найти IDOR вручную

Не входя в систему, запросите профиль администратора:

```
curl -s "http://127.0.0.1:5000/profile?id=1" | grep -E "Email|заметка"
```

Ожидаемый результат: почта администратора и его закрытая заметка. Ни Semgrep, ни Trivy, ни ZAP об этом не сообщили.

Откройте `backend/app.py`, найдите функцию `profile` и ответьте: какой проверки в ней не хватает?

В отчёт: команда, её вывод, объяснение причины.

Шаг А4. Исправить контроль доступа

Все правки — в `backend/app.py`.

1. В первой строке с импортом Flask добавьте `abort`:

```
from flask import Flask, abort, g, redirect, render_template, request, session
```

2. В функции `login` после строки `session["username"] = user[1]` добавьте строку с тем же отступом:

```
                session["role"] = user[2]
```

Теперь сервер помнит роль вошедшего пользователя.

3. В функции `profile` замените строку

```
        user_id = request.args.get("id", "1")
```

на

```
        if "user_id" not in session:
            abort(401)
        user_id = request.args.get("id", default=session["user_id"], type=int)
        if user_id != session["user_id"] and session.get("role") != "admin":
            abort(403)
```

Логика: нет входа — 401. Если `id` не указан, показываем собственный профиль. Чужой профиль доступен только администратору, остальным — 403.

4. В файле `frontend/templates/base.html` замените ссылку `<a href="/profile?id=2">Профиль</a>` на `<a href="/profile">Профиль</a>`.

Шаг А5. Добавить заголовки безопасности

В `backend/app.py` найдите строку `database.init_app(app)` и сразу после неё вставьте (отступ — четыре пробела, как у соседних строк):

```
    @app.after_request
    def security_headers(response):
        response.headers["Content-Security-Policy"] = (
            "default-src 'self'; frame-ancestors 'none'; base-uri 'self'; form-a
ction 'self'"
        )
        response.headers["X-Content-Type-Options"] = "nosniff"
        response.headers["X-Frame-Options"] = "DENY"
        response.headers["Permissions-Policy"] = "camera=(), microphone=(), geol
ocation=()"
        return response
```

Функция с пометкой `@app.after_request` выполняется после обработки каждого запроса и дописывает заголовки в ответ.

Шаг А6. Написать тесты на контроль доступа

В `tests/test_app.py` **замените** функцию `test_profile_is_available_without_authentication` на:

```
def login(client, username, password):
    return client.post("/login", data={"username": username, "password": passwor
d})


def test_profile_requires_login(client):
    assert client.get("/profile?id=1").status_code == 401


def test_student_cannot_read_foreign_profile(client):
    login(client, "student", "student123")
    assert client.get("/profile?id=1").status_code == 403


def test_student_reads_own_profile(client):
    login(client, "student", "student123")
    response = client.get("/profile?id=2")
    assert response.status_code == 200
    assert "student@example.test" in response.get_data(as_text=True)


def test_admin_reads_any_profile(client):
    login(client, "admin", "admin123")
    assert client.get("/profile?id=3").status_code == 200


def test_security_headers_are_set(client):
    headers = client.get("/").headers
    assert "default-src 'self'" in headers["Content-Security-Policy"]
    assert headers["X-Content-Type-Options"] == "nosniff"
    assert headers["X-Frame-Options"] == "DENY"
```

Эти тесты — запись бизнес-правила на языке, понятном машине. Сканер не знал, кому какой профиль можно видеть; теперь это знает конвейер.

```
pytest -q
```

Ожидаемый результат: `11 passed`.

В отчёт: изменения кода (`git diff backend/app.py`) и вывод `pytest`.

Шаг А7. Повторить сканирование

```
docker compose up -d --build --wait
curl -s -o /dev/null -w "%{http_code}\n" "http://127.0.0.1:5000/profile?id=1"
curl -sI http://127.0.0.1:5000/ | grep -iE "content-security|x-content|x-frame|permissions"
docker run --rm --add-host host.docker.internal:host-gateway -v "$PWD/reports:/zap/wrk/:rw" ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t http://host.docker.internal:5000 -r zap-after.html -I
docker compose down -v
```

Ожидаемый результат:

- профиль без входа: `401`;
- четыре заголовка безопасности в ответе;
- ZAP: `WARN-NEW: 6` вместо 10.

Разберите шесть оставшихся предупреждений и для каждого примите решение:

Код	Название	Решение	Обоснование
10202	Absence of Anti-CSRF Tokens	настоящая проблема, в план доработок	формы не защищены от подделки запросов; требуется отдельная доработка
10036	Server Leaks Version Information		
10031	User Controllable HTML Element Attribute		
90004	Cross-Origin-Embedder-Policy Header Missing		
10049	Non-Storable Content		
10111	Authentication Request Identified		

Варианты решения: «исправить сейчас», «в план доработок», «информационное, действий не требует», «ложное срабатывание» — каждое с обоснованием. Подсказка к 10031: ZAP заметил, что текст запроса попадает в атрибут `value` поля поиска. Проверьте в исходном коде страницы, экранируется ли он.

Сохраните результат:

```
git add -A
git commit -m "lab5: access control for profiles and security headers"
```

В отчёт: сравнение «10 → 6», таблица решений.

Часть Б. Конвейер в GitHub Actions

Шаг Б1. Опубликовать репозиторий на GitHub

1. Войдите на https://github.com, нажмите **+ → New repository**.
2. Имя: `devsecops-lab`. Видимость: **Public** (для публичных репозиториев GitHub Actions бесплатны и доступна защита веток).
3. **Не** отмечайте создание README, `.gitignore` и лицензии — репозиторий должен быть пустым.
4. Нажмите **Create repository**.

Перед публикацией последний раз проверьте, что в репозитории нет секретов:

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 git --redact -v
 /src
```

Ожидаемый результат: `no leaks found`.

Свяжите локальный репозиторий с GitHub и отправьте код (подставьте своё имя пользователя):

```
git remote add origin https://github.com/<ваш_логин>/devsecops-lab.git
git push -u origin main
```

При первой отправке Git попросит войти в GitHub — откроется окно браузера. Если вместо этого в терминале запрашивается пароль, нужен персональный токен доступа: **Settings → Developer settings → Personal access tokens**. Вводите его только в ответ на запрос пароля. Не вставляйте токен в адрес репозитория и не сохраняйте в файлах проекта — вы знаете почему.

Ожидаемый результат: на странице репозитория видны ваши файлы и шесть коммитов.

В отчёт: ссылка на репозиторий.

Шаг Б2. Первый конвейер: тесты

Создайте каталог и файл:

```
mkdir -p .github/workflows
```

Файл `.github/workflows/security.yml`:

```
name: security-pipeline

on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:

permissions:
  contents: read
```

```yaml
jobs:
  tests:
    name: Unit tests
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip
      - name: Install dependencies
        run: python -m pip install -r requirements.txt -r requirements-dev.txt
      - name: Run tests
        run: pytest -q
```

Разбор:

- on — когда запускать: при отправке в `main`, при любом pull request и вручную;
- `permissions: contents: read` — конвейеру разрешено только читать код;
- `runs-on: ubuntu-latest` — задание выполняется на чистой виртуальной машине с Linux;
- `actions/checkout@v4` — получить код репозитория;
- `actions/setup-python@v5` — установить Python;
- `run` — обычные команды терминала, те же, что вы выполняли на ноутбуке.

В YAML отступы значимы: только пробелы, по два на уровень.

```
git add .github/workflows/security.yml
git commit -m "ci: add unit tests job"
git push
```

Откройте на GitHub вкладку **Actions**. Через минуту-две у запуска появится зелёная отметка. Щёлкните по запуску, затем по заданию `Unit tests` и найдите в журнале строку `11 passed`.

Если запуск красный — откройте журнал упавшего шага. Чаще всего причина в отступах YAML.

В отчёт: снимок экрана зелёного запуска.

Шаг Б3. Задание: поиск секретов

Добавьте в конец файла второе задание. Слово `secrets:` должно стоять на том же уровне отступа, что и `tests:` (два пробела).

```yaml
  secrets:
    name: Secrets (Gitleaks)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
        with:
          fetch-depth: 0
      - name: Install Gitleaks
        run: |
          curl -sSfL https://github.com/gitleaks/gitleaks/releases/download/v8.2
4.2/gitleaks_8.24.2_linux_x64.tar.gz \
            | sudo tar -xz -C /usr/local/bin gitleaks
      - name: Scan Git history
        run: |
          mkdir -p reports
          gitleaks git --redact --no-banner --report-format json --report-path r
eports/gitleaks.json .
      - name: Upload report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: gitleaks-report
          path: reports/
```

Разбор:

- `fetch-depth: 0` — получить **всю** историю. По умолчанию checkout берёт только последний коммит, и проверять историю было бы не на чем;
- версия Gitleaks та же, что в `.pre-commit-config.yaml`: локальная и серверная проверки должны совпадать;
- `if: always()` — сохранить отчёт, даже если предыдущий шаг упал. Отчёт нужнее всего именно тогда, когда проверка не пройдена;
- файл `.gitleaksignore` из ЛР3 учитывается автоматически.

Шаг Б4. Задание: статический анализ

```
  sast:
    name: SAST (Semgrep)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip
      - name: Install Semgrep
        run: python -m pip install -r requirements-dev.txt
      - name: Run Semgrep
        run: |
          mkdir -p reports
          semgrep scan --config .semgrep.yml --metrics=off --error --json-output
 reports/semgrep.json .
      - name: Upload report
        if: always()
        uses: actions/upload-artifact@v4
        with:
```

```
          name: semgrep-report
          path: reports/
```

Параметр `--error` — тот самый, что вы проверяли в ЛР2: любая находка даёт ненулевой код, и задание становится красным.

Шаг Б5. Задание: зависимости и SBOM

```
  sca:
    name: SCA and SBOM (Trivy)
    runs-on: ubuntu-latest
    env:
      TRIVY_IMAGE: aquasec/trivy:0.76.2
    steps:
      - uses: actions/checkout@v4
      - name: Scan dependencies
        run: |
          mkdir -p reports && chmod 777 reports
          docker run --rm -v "$PWD:/src" "$TRIVY_IMAGE" fs --scanners vuln \
            --severity HIGH,CRITICAL --exit-code 1 /src
      - name: Check Dockerfile
        run: docker run --rm -v "$PWD:/src" "$TRIVY_IMAGE" config --exit-code 1
/src
      - name: Generate SBOM
        run: |
          docker run --rm -v "$PWD:/src" "$TRIVY_IMAGE" fs --format cyclonedx \
            --output /src/reports/sbom.cdx.json /src
      - name: Upload SBOM
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: sbom
          path: reports/
```

Здесь реализованы правила приёмки из ЛР4: HIGH и CRITICAL в зависимостях и любая находка в Dockerfile блокируют изменение. SBOM сохраняется как артефакт каждого запуска.

Отправьте три новых задания и убедитесь, что все четыре зелёные:

```
git add -A
git commit -m "ci: add Gitleaks, Semgrep and Trivy jobs"
git push
```

На странице запуска внизу, в разделе **Artifacts**, появились три архива. Скачайте `sbom` и убедитесь, что внутри тот же формат, что вы получали в ЛР4.

В отчёт: снимок экрана запуска с четырьмя заданиями и разделом Artifacts.

Шаг Б6. Задание: запуск приложения и DAST

Сначала зафиксируйте решения из шага А7 в файле правил. Создайте в корне проекта файл `zap-rules.tsv`. Колонки разделяются **знаком табуляции**, а не пробелами; чтобы не ошибиться, создайте файл командой:

```
printf '10038\tFAIL\t(Content-Security-Policy)\n10020\tFAIL\t(Anti-clickjacking)\n10021\tFAIL\t(X-Content-Type-Options)\n10202\tWARN\t(Anti-CSRF: backlog)\n10049\tIGNORE\t(informational)\n10111\tIGNORE\t(informational)\n' > zap-rules.tsv
cat zap-rules.tsv
```

Смысл: отсутствие любого из трёх заголовков, которые вы добавили, — провал (`FAIL`); отсутствие защиты от CSRF остаётся предупреждением до доработки; два информационных сообщения скрыты. Если кто-нибудь удалит заголовки, конвейер это заметит.

Добавьте в `security.yml` последнее задание:

```yaml
  dast:
    name: Container and DAST (ZAP)
    runs-on: ubuntu-latest
    needs: [tests, secrets, sast, sca]
    steps:
      - uses: actions/checkout@v4
      - name: Start application
        run: docker compose up -d --build --wait
      - name: Smoke test
        run: curl --fail --silent http://127.0.0.1:5000/health
      - name: ZAP baseline scan
        run: |
          mkdir -p reports
          cp zap-rules.tsv reports/
          chmod -R 777 reports
          docker run --rm --network host -v "$PWD/reports:/zap/wrk/:rw" \
            ghcr.io/zaproxy/zaproxy:stable zap-baseline.py \
            -t http://127.0.0.1:5000 -c zap-rules.tsv -r zap-report.html -I
      - name: Stop application
        if: always()
        run: docker compose down -v
      - name: Upload ZAP report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: zap-report
          path: reports/
```

Разбор:

- `needs: […]` — задание запускается только после успеха четырёх предыдущих. Нет смысла поднимать приложение и тратить время на DAST, если в коде уже нашли секрет или инъекцию. Так получается порядок «сначала быстрые и дешёвые проверки, потом медленные»;

- `docker compose up -d --build --wait` — собрать образ, запустить приложение с базой и дождаться готовности;
- `-c zap-rules.tsv` — применить ваши правила; с ними `-I` пропускает предупреждения, но `FAIL` по-прежнему останавливает конвейер.

```
git add -A
git commit -m "ci: add container start and ZAP baseline with rules"
git push
```

Ожидаемый результат: в запуске пять заданий, все зелёные; `dast` стартует после остальных. На странице запуска виден граф заданий. Общее время — 3–5 минут.

В отчёт: снимок экрана графа заданий, итоговая строка ZAP из журнала, полный текст `security.yml`.

Шаг Б7. Опыт: конвейер останавливает уязвимое изменение

Конвейер, который ни разу не «покраснел», ещё ничего не доказал. Сыграйте роль разработчика, который в спешке вносит уязвимость.

```
git switch -c feature/rich-comments
```

В файле `frontend/templates/comments.html` верните фильтр: замените `{{ comment[2] }}` на `{{ comment[2]|safe }}`.

```
git add -A
git commit -m "Allow rich text in comments"
git push -u origin feature/rich-comments
```

Локальный хук коммит пропустил: он ищет секреты, а не XSS.

На GitHub откройте репозиторий: появится кнопка **Compare & pull request**. Создайте pull request в ветку `main`.

Ожидаемый результат через 1–2 минуты:

- задания `Unit tests` и `SAST (Semgrep)` — красные;
- задание `Container and DAST (ZAP)` — пропущено (skipped): сработало условие `needs`;
- внизу страницы PR сообщение о непройденных проверках.

Откройте журнал `SAST (Semgrep)` и найдите правило и строку. Откройте журнал `Unit tests` и найдите упавший тест.

Теперь исправьте ошибку в той же ветке: верните `{{ comment[2] }}`, затем

```
git add -A
git commit -m "Revert unsafe filter"
git push
```

PR перепроверится автоматически и станет зелёным. Нажмите **Merge pull request**, затем вернитесь в основную ветку:

```
git switch main
git pull
```

В отчёт: снимок экрана красного PR, фрагменты журналов Semgrep и pytest с причиной, снимок экрана зелёного PR после исправления. Ответ на вопрос: уязвимость поймали сразу две проверки — какая из них сработала бы, если бы вы забыли написать тест в ЛР2?

Шаг Б8. Сделать проверки обязательными

Сейчас красный PR всё ещё можно слить — кнопка лишь предупреждает. Закройте эту возможность.

На GitHub: **Settings → Branches → Add branch ruleset** (или **Add classic branch protection rule**):

1. Имя: `protect-main`; Enforcement status: **Active**.
2. Target branches: **Include default branch**.
3. Включите **Require a pull request before merging**.
4. Включите **Require status checks to pass** и добавьте проверки: `Unit tests`, `Secrets (Gitleaks)`, `SAST (Semgrep)`, `SCA and SBOM (Trivy)`, `Container and DAST (ZAP)`.
5. Сохраните.

Проверьте: попытка `git push` напрямую в `main` теперь отклоняется, а PR с красной проверкой нельзя слить.

В отчёт: снимок экрана настроенного правила.

Шаг Б9. Итог практикума: решение о выпуске

Вернитесь к таблице требований из ЛР1 и заполните итоговую:

№	Требование	Статус	Чем подтверждено	Проверяется в CI
T1	SQL только через параметры	выполнено	тесты, Semgrep: 0 находок	да: `tests`, `sast`
…				

Затем составьте реестр остаточных рисков — того, что осталось **не** исправленным. Как минимум:

Риск	Почему остался	Что нужно сделать
Пароли хранятся в открытом виде	ни один сканер не сообщил; не входило в работы	хранить хеши (например, `werkzeug.security`)
Нет защиты от CSRF	ZAP 10202, оставлено предупреждением	добавить токены в формы
Ключ сессии по умолчанию `dev-only-change-me`		

Риск	Почему остался	Что нужно сделать
Уязвимости базового образа без исправлений		
Приложение запускается встроенным сервером разработки		

И ответьте на главный вопрос: **можно ли выпускать это приложение в эксплуатацию?** Конвейер зелёный — достаточно ли этого? Обоснуйте в 5–7 предложениях.

В отчёт: таблица требований, реестр остаточных рисков, решение о выпуске.

3. Дополнительные задания

Выполняются по желанию, оцениваются преподавателем отдельно.

1. **Хеширование паролей.** Храните в базе не пароль, а результат `generate_password_hash`; при входе сравнивайте через `check_password_hash`. Обновите тесты.
2. **Автоматическое обновление зависимостей.** Добавьте файл `.github/dependabot.yml` для экосистем `pip` и `github-actions` и покажите созданный Dependabot pull request.
3. **Сканирование образа в CI.** Добавьте в задание `dast` шаг `trivy image` с сохранением отчёта как артефакта, без блокировки. Объясните, почему без блокировки.
4. **Активное сканирование.** Только против приложения на собственном компьютере выполните `zap-full-scan.py` на копии проекта из первого коммита и сравните результаты с baseline.

4. Требования к отчёту

Часть А

1. Итог первого сканирования ZAP, таблица десяти предупреждений, ответы на вопросы о покрытии.
2. Доказательство IDOR и объяснение причины.
3. Изменения кода и тесты, вывод `pytest`.
4. Результат повторного сканирования, таблица решений по оставшимся предупреждениям.

Часть Б

5. Ссылка на репозиторий GitHub.
6. Полный текст `security.yml` и `zap-rules.tsv`.
7. Снимки экрана: зелёный запуск с пятью заданиями и артефактами; красный PR; зелёный PR; правило защиты ветки.
8. Разбор опыта с уязвимым изменением.

9. Итоговая таблица требований, реестр остаточных рисков, решение о выпуске.
10. Общий вывод по практикуму.

5. Контрольные вопросы

1. Чем DAST отличается от SAST? Приведите по одному дефекту из практикума, который находит только один из них.
2. Чем пассивное сканирование отличается от активного? Почему активное нельзя запускать против чужих систем?
3. Почему `FAIL: 0` в отчёте ZAP не доказывает безопасность?
4. Почему IDOR не находят автоматические сканеры? Чем его ловить?
5. В чём разница между кодами ответа 401 и 403?
6. Как GitHub Actions определяет, что шаг провален?
7. Зачем заданию `secrets` параметр `fetch-depth: 0`?
8. Зачем заданию `dast` параметр `needs`? Что изменится, если его убрать?
9. Зачем нужен `if: always()` у шага сохранения отчёта?
10. Почему локального хука pre-commit недостаточно и ту же проверку повторяют в CI?
11. Почему в конвейере указаны точные версии сканеров и `permissions: contents: read`?
12. Конвейер зелёный. Означает ли это, что приложение безопасно и соответствует ГОСТ Р 56939-2024?

6. Критерии оценивания

Критерий	Баллы
Часть А. Сканирование ZAP и разбор предупреждений	10
Обнаружение и исправление IDOR, заголовки, тесты	20
Повторное сканирование и решения по остаткам	10
Часть Б. Репозиторий на GitHub, задание с тестами	10
Задания Gitleaks, Semgrep, Trivy, артефакты	15
Задание DAST с файлом правил и зависимостью `needs`	10
Опыт с уязвимым изменением: красный и зелёный PR	10
Защита ветки	5
Итоговая таблица требований, остаточные риски, решение о выпуске	10
Итого	**100**

7. Источники

- ГОСТ Р 56939-2024. Разработка безопасного программного обеспечения. Общие требования.
- Документация GitHub Actions: https://docs.github.com/actions
- Защита веток в GitHub: https://docs.github.com/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/about-rulesets

- OWASP ZAP, baseline scan: https://www.zaproxy.org/docs/docker/baseline-scan/
- OWASP Top 10, A01 Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- OWASP Secure Headers Project: https://owasp.org/www-project-secure-headers/